

Complete DSA Learning Order

Topic Content Compilation

Section 12 — Greedy Algorithms (12.1 – 12.4)

June 30, 2026

Section 12 — Greedy Algorithms

12.1 — The greedy choice property — what makes greedy valid for a problem

1. PLAIN-LANGUAGE GIST

The greedy choice property is what makes a “always pick the locally best option, never look back” strategy actually produce the globally optimal answer — and the entire skill of this topic is recognizing that “greedy” isn’t a universally applicable technique, but a specific property a problem either does or doesn’t have, requiring justification rather than just intuition.

2. REAL-WORLD ANALOGY

Picture making change for a customer using the fewest coins possible, always handing over the largest coin that doesn’t exceed the remaining amount owed. For standard currency denominations (quarters, dimes, nickels, pennies), this greedy “always take the biggest coin that fits” strategy genuinely produces the fewest possible coins, every time. But change this single detail — imagine a currency with coins worth 25, 10, and 1 (no nickel) — and that same greedy strategy can produce a worse answer than a smarter combination, even though the strategy “felt” just as reasonable.

3. CONNECT TO PRIOR KNOWLEDGE

This directly parallels 11.1’s DP-property recognition skill, but for a different (related) family of techniques — just as 11.1 taught you to verify overlapping subproblems and optimal substructure before reaching for memoization/tabulation, this topic teaches you to verify the greedy choice property before reaching for a greedy strategy, rather than assuming any “obviously sensible” local rule will automatically produce a correct global answer.

4. CORE MECHANISM (Simple → Intermediate)

Simple model: A problem has the greedy choice property if, at every decision point, choosing the option that looks best right now (by some specific, problem-relevant measure) is guaranteed to be part of some optimal overall solution — meaning you never need to reconsider or undo that choice later, no matter what subsequent choices turn out to be.

Next layer — why this property must be proven, not assumed, and how greedy failure actually looks in practice:

- **The property is specific to the exact problem and the exact greedy rule being proposed** — as the coin-change example shows, the same greedy rule (“take the largest available option”) is valid for one set of coin denominations and invalid for another; the property isn’t a fact about “greedy thinking” in the abstract, it’s a fact about the combination of a specific problem and a specific proposed rule.
- **When greedy fails, it doesn’t fail loudly** — a greedy algorithm applied to a problem that lacks the greedy choice property still runs, still terminates, and still produces an answer; it just isn’t guaranteed to be the optimal one. This is precisely why verification matters: an incorrect greedy approach can pass many test cases while silently failing on others.
- **Common ways to actually verify the property:** constructing a small counterexample by hand is often the fastest practical check; a more rigorous approach uses an exchange argument (briefly previewed in 11.1’s deeper knowledge) — assuming a hypothetical optimal solution that doesn’t make the greedy choice, then showing you can always “exchange” some part of it to incorporate the greedy choice without making the solution worse.

- **The greedy choice property is closely related to, but distinct from, optimal substructure (11.1)** — both are about whether a problem decomposes cleanly, but greedy additionally requires that you never need to compare multiple candidate choices and keep several possibilities alive (the way DP's table tracks many subproblem answers simultaneously) — greedy commits to exactly one choice at each step, irrevocably.

5. VISUAL REPRESENTATION

Greedy choice property: locally best choice leads to a globally optimal answer

Making change for 30 cents with coins [25, 10, 5, 1] -- greedy: always take the LARGEST coin that fits

Step 1: 30 cents remaining -- take 25 (largest that fits) -> remaining = 5

Step 2: 5 cents remaining -- take 5 -> remaining = 0

Result: 2 coins [25, 5] -- this happens to BE the optimal answer

Same greedy strategy with coins [25, 10, 1] FAILS for 30 cents:

Step 1: 30 cents -- take 25 -> remaining = 5

Step 2: 5 cents -- no 5-coin exists -- take 1, five times -> 5 more coins

Result: 6 coins [25,1,1,1,1,1] -- but [10,10,10] uses only 3!

The SAME greedy rule (take the largest coin that fits) is valid for one coin set but INVALID for another -- greedy validity depends on the problem's specific structure, not just on "the greedy idea sounding reasonable."

The diagram traces making 30 cents of change with two different coin sets: with [25, 10, 5, 1], the greedy "take the largest coin that fits" strategy produces the genuinely optimal 2-coin answer; with [25, 10, 1] (no nickel), the exact same greedy strategy produces a 6-coin answer, while a smarter combination ([10, 10, 10]) only needs 3 — the same rule, valid for one problem instance, invalid for another.

6. WHEN TO USE / WHEN NOT TO USE

Apply greedy reasoning when:

- You can construct (or recall) a proof — an exchange argument, or at minimum a convincing inability to find a counterexample after genuinely trying — that the locally best choice is always safe to commit to without reconsideration.
- The problem is a recognized instance of a pattern with an already-established greedy proof (12.3's interval scheduling, 12.4's common templates) — recognizing the pattern transfers the existing proof, rather than needing to construct one from scratch every time.

Wrong choice / red flag when:

- You've proposed a greedy rule based purely on "it seems like the obviously sensible choice" without attempting to find a counterexample — this is precisely the trap the coin-change example demonstrates.
- A quick attempt to construct a counterexample succeeds — if you can find even one input where the greedy rule's chosen path provably leads to a worse final answer than some alternative, the greedy approach is invalid for that problem, full stop, and a different technique (often DP, 12.2's focus) is needed instead.

7. C# EXAMPLES — Simple → Intermediate → Advanced

SIMPLE — greedy coin change, valid for standard denominations, the exact diagram's first scenario:

```
List<int> GreedyCoinChange(int[] coins, int amount) {
    // assumes coins is sorted descending, e.g. [25, 10, 5, 1]
    var result = new List<int>();
    foreach (int coin in coins) {
        while (amount >= coin) {
            result.Add(coin);
            amount -= coin;
        }
    }
    return amount == 0 ? result : null; // null = couldn't make change
}
```

INTERMEDIATE — demonstrating the exact same function failing on a non-canonical coin set, the diagram's second scenario:

```
// Same GreedyCoinChange function, different coins:
var badCoins = new[] { 25, 10, 1 }; // sorted descending, NO nickel
var result = GreedyCoinChange(badCoins, 30);
// result = [25, 1, 1, 1, 1, 1] -- 6 coins

// But the TRUE optimal answer:
var optimal = new[] { 10, 10, 10 }; // only 3 coins
// GreedyCoinChange cannot find this -- it commits to 25 immediately
// and never reconsiders, even though that commitment was a mistake
```

This is the same function from the simple example, run on different input — explicitly demonstrating that the code itself isn't “wrong” in a bug sense; the strategy is invalid for this specific input set, a structural mismatch rather than an implementation error.

ADVANCED — verifying the greedy choice property via a brute-force comparison, a practical sanity-check technique for new problems:

```
// When uncertain whether a greedy rule is valid, compare its output
// against an exhaustive brute-force search on SMALL test cases --
// if they ever disagree, the greedy rule is proven invalid.
int GreedyCoinCount(int[] coins, int amount) {
    var result = GreedyCoinChange(coins, amount);
    return result?.Count ?? int.MaxValue;
}

int BruteForceCoinCount(int[] coins, int amount, Dictionary<int,int> memo) {
    if (amount == 0) return 0;
    if (amount < 0) return int.MaxValue;
    if (memo.TryGetValue(amount, out int cached)) return cached;

    int best = int.MaxValue;
    foreach (int coin in coins) {
        int sub = BruteForceCoinCount(coins, amount - coin, memo);
        if (sub != int.MaxValue) best = Math.Min(best, sub + 1);
    }
    return memo[amount] = best;
}

// Compare GreedyCoinCount against BruteForceCoinCount across many
// small (coins, amount) combinations -- any disagreement is a
// concrete, discovered counterexample.
```

This shows a genuinely practical engineering technique: rather than trying to prove or disprove the greedy choice property purely through abstract reasoning, you can empirically test it by comparing against a brute-force (or DP) solution across many small cases — a disagreement is a definitive counterexample, while agreement across many diverse small cases is reasonably strong supporting evidence.

8. DEEPER KNOWLEDGE (Gist Only)

At an expert level, problems that genuinely have the greedy choice property often also have a deeper structural property called matroid structure (from a branch of combinatorics) — a formal mathematical framework that, when it applies, guarantees greedy algorithms work correctly, providing a principled alternative to ad hoc counterexample-hunting. There's also a useful practical heuristic worth knowing: greedy algorithms that are valid tend to have proofs following one of a small number of recurring argument shapes (exchange arguments, “greedy stays ahead” arguments, or matroid-based arguments) — recognizing which shape a new problem's proof would need to take is itself a learnable skill that becomes faster with exposure to more examples.

9. COMMON MISCONCEPTIONS AND MISTAKES

A common mistake is assuming any locally-sensible-sounding rule must be a valid greedy strategy without attempting to verify it — “obviously, taking the biggest available option is best” is exactly the kind of unverified intuition that fails on the modified coin-change example. Another is conflating “the greedy algorithm runs and produces an answer” with “the greedy algorithm produces the correct answer” — a flawed greedy approach doesn't error out or crash, it just silently returns a suboptimal result. A third: assuming that because a greedy approach works on the specific test cases you've tried, it must be generally valid — passing several test cases is weak evidence at best; genuinely searching for a counterexample is the only way to be confident.

10. SELF-CHECK

- Why does the exact same greedy rule (“take the largest available option”) work correctly for one set of coin denominations but fail for another — what's different about the underlying problem structure?
- What's the practical difference between “this greedy algorithm ran successfully and returned an answer” and “this greedy algorithm returned the correct answer” — why doesn't the first imply the second?
- Describe, in your own words, what an exchange argument is trying to prove, and why successfully constructing one would justify trusting a greedy approach.

12.2 — Recognizing when greedy works vs when it gives a wrong answer (vs needing DP)

1. PLAIN-LANGUAGE GIST

The practical test for whether a problem can be solved greedily, rather than needing the fuller machinery of dynamic programming, is asking: after making the locally best choice, could any later decision ever reveal that the earlier choice should have been different? If the answer is provably no, greedy applies and is typically much simpler and faster than DP; if the answer is yes (or even just possibly yes), DP's "keep multiple possibilities alive and compare them" approach is the safer, often necessary choice.

2. REAL-WORLD ANALOGY

Picture choosing a route through a one-way maze where every decision point has a sign telling you definitively which direction leads to the exit fastest — you can trust the sign and never need to backtrack, exactly greedy's irrevocable-commitment style. Now picture a maze with no such reliable signs, where the best-looking path at each junction sometimes turns out to be a dead end, requiring you to explore multiple paths and compare them before committing — that's DP's "keep several possibilities alive and compare" style instead.

3. CONNECT TO PRIOR KNOWLEDGE

This topic directly operationalizes 12.1's abstract greedy choice property into a concrete decision procedure, and it explicitly connects back to Tier 1's entire DP section (11.1-11.7) — greedy and DP aren't unrelated techniques competing for the same problems; greedy is best understood as the special case of DP where, at every step, only one candidate choice ever needs to be tracked forward, because the others are provably never going to be better, collapsing DP's "track every possibility" table down to a single running value.

4. CORE MECHANISM (Simple → Intermediate)

Simple model: For a given problem, examine the structure of the decision being made at each step. If choosing the locally optimal option at each step can be proven to never require reconsideration later, the problem reduces to a simple, fast greedy pass. If the locally optimal choice might need to be reconsidered depending on choices made later, the problem requires DP's broader "compare all viable possibilities" approach instead.

Next layer — concrete signals pointing toward each technique, and why DP is the "fallback" when greedy's proof fails:

- **Signals favoring greedy:** the problem involves sorting by some natural criterion and then making a single pass of irrevocable decisions (12.3's interval scheduling, 12.4's common templates); a proof or strong intuition exists that the locally best choice is always safe; the problem has a simple, single-dimensional "running state" that doesn't need to track multiple coexisting possibilities.
- **Signals favoring DP:** the problem involves selecting a combination of items where an early choice's value depends on interactions with potential later choices (11.6's knapsack is the canonical example); a greedy attempt produces a counterexample (12.1's verification process) when tested.
- **Why DP is the conceptual fallback, not an unrelated alternative:** DP's tables exist specifically to track "the best answer for every possible sub-state," precisely because, in DP-requiring problems, you genuinely don't know in advance which sub-state will turn out to matter for the final optimal answer — greedy's single-path commitment is only safe when that uncertainty doesn't exist.

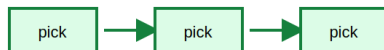
- **A practical diagnostic procedure:** attempt a greedy solution first (since it's typically simpler to write and reason about), then actively try to break it with a hand-constructed counterexample (12.1's technique) — if you find one, you've confirmed DP is needed and have also likely gained insight into the problem's actual structure that will help you design the correct DP state.

5. VISUAL REPRESENTATION

Greedy vs DP: does the locally-best choice ever need reconsidering?

Greedy works: Activity Selection

picking the activity that ends EARLIEST
never needs to be undone later



no backtracking, ever -- straight line of decisions

DP needed: 0/1 Knapsack

picking the highest-value item first
can leave no room for a better COMBINATION



must compare ALL combinations -- exactly DP's job

The test: after making the greedy choice, can a LATER decision ever reveal that the earlier choice should have been different?

If NO (the choice is provably safe, never reconsidered) -- GREEDY applies, $O(n \log n)$ typical.

If YES (the optimal answer might require a DIFFERENT earlier choice) -- need DP, exploring/comparing multiple possibilities rather than committing irrevocably.

Greedy is a SPECIAL CASE where DP's "compare all subproblem combinations" collapses into "always pick the single best option" -- it's not a separate, unrelated technique.

The diagram contrasts activity selection (where picking by earliest end time is a single, irrevocable, always-safe chain of decisions — greedy applies cleanly) against 0/1 knapsack (where an early high-value pick can block a better later combination, requiring DP's "keep multiple possibilities alive" comparison) — the structural difference between "never needs undoing" and "might need undoing" made visually concrete.

6. WHEN TO USE / WHEN NOT TO USE

Reach for greedy first when:

- The problem matches a recognized greedy-friendly pattern (12.3, 12.4) or you can quickly construct a convincing exchange-argument-style justification — greedy solutions are typically simpler to implement and faster (often $O(n \log n)$ from sorting) than the equivalent DP solution.

Fall back to DP when:

- A genuine attempt to find a greedy counterexample succeeds, or you can identify a specific scenario where an early "locally best" choice would need to be reconsidered based on later information.
- The problem explicitly involves combinatorial selection with interacting constraints (capacity limits, mutual exclusivity beyond simple non-overlap) — this is frequently a DP-requiring shape, even when a tempting greedy heuristic exists.

7. C# EXAMPLES — Simple → Intermediate → Advanced

SIMPLE — a problem where greedy is provably correct, contrasted explicitly with one where it isn't:

```
// GREEDY WORKS: maximize value with NO capacity constraint at all --
// just take every item with positive value, no interaction
int MaxValueNoCapacity(int[] values) {
    int total = 0;
    foreach (int v in values) {
        if (v > 0) total += v;    // each choice is fully independent
    }
    return total;
}

// GREEDY FAILS: same "take high value" instinct, but NOW there's a
// capacity constraint creating interaction between choices (11.6's
// knapsack) -- picking greedily by value can leave wasted capacity.
```

INTERMEDIATE — explicitly testing a proposed greedy heuristic against DP on a small knapsack instance, demonstrating the failure concretely:

```
// Greedy heuristic: sort by value descending, take what fits
int GreedyKnapsackAttempt(int[] weights, int[] values, int capacity) {
    var items = Enumerable.Range(0, weights.Length)
        .OrderByDescending(i => values[i]).ToArray();
    int totalValue = 0, remaining = capacity;
    foreach (int i in items) {
        if (weights[i] <= remaining) {
            totalValue += values[i];
            remaining -= weights[i];
        }
    }
    return totalValue;
}

// Test case: weights = [5, 4, 4], values = [10, 9, 9], capacity = 8
// Greedy picks item 0 (value 10, weight 5) -- remaining capacity 3,
// neither remaining item (weight 4 each) fits -- greedy total = 10
// True optimal: items 1+2 (value 9+9=18, weight 4+4=8, fits exactly)
// Greedy (10) < Optimal (18) -- CONFIRMED counterexample
```

This directly applies 12.1's “construct a counterexample” verification technique to a specific, concrete instance — showing not just that greedy can fail on knapsack-style problems in the abstract, but a specific, checkable numeric example where it demonstrably does.

ADVANCED — a problem that initially looks like it might need DP, but a careful greedy proof reveals it doesn't (job sequencing to maximize count of jobs completed by their deadlines):

```
// Each job has a deadline; only one job can run per time unit.
// Maximize the NUMBER of jobs completed by their deadline.
int MaxJobsByDeadline(int[] deadlines) {
    Array.Sort(deadlines);    // earliest deadlines first
    int slotsUsed = 0;
    int time = 0;
    foreach (int deadline in deadlines) {
        if (time < deadline) {    // still time to schedule this job
            time++;
            slotsUsed++;
        }
        // else: can't make this deadline, skip it -- no later
        // reconsideration ever helps
    }
    return slotsUsed;
}
```


This is deliberately included to show the other direction of the diagnostic process: a problem that superficially resembles a “scheduling with constraints” shape (which can sometimes need DP) but, upon closer analysis, turns out to have a valid, provable greedy solution — reinforcing that the diagnostic process applies in both directions, not just as a way to justify reaching for DP.

8. DEEPER KNOWLEDGE (Gist Only)

At an expert level, some problems exhibit a middle ground sometimes called “greedy with exchange/repair” — an initial greedy solution is computed quickly, then a follow-up local-search or repair phase fixes specific known failure modes, achieving near-optimal (though not always provably optimal) results faster than full DP for certain large-scale practical problems. There's also a valuable historical/theoretical note: many classic “greedy works” proofs (activity selection, Huffman coding, minimum spanning tree algorithms like Kruskal's and Prim's) were significant results precisely because it wasn't obvious in advance that greedy would work — reinforcing that “does greedy work here” is a real, substantive question, not a triviality.

9. COMMON MISCONCEPTIONS AND MISTAKES

A common mistake is assuming that because a problem involves “picking the best available option,” it must be solvable greedily — many DP problems also involve picking among options, but the presence of an interacting constraint is what actually determines whether greedy's irrevocability is safe. Another is giving up on greedy too early, assuming any problem with multiple constraints automatically needs DP — the job-sequencing example shows that some constrained problems still admit a clean, provable greedy solution. A third: treating “I tried greedy and it passed my test cases” as sufficient validation — the diagnostic process specifically requires actively trying to construct a counterexample, not just passively hoping one doesn't exist.

10. SELF-CHECK

- State the core diagnostic question that determines whether a problem needs greedy or DP, in your own words, without referencing a specific example problem.
- Why does maximizing total value with no capacity constraint admit a trivial greedy solution, while the same “pick high value first” instinct fails once a shared capacity constraint is introduced?
- In the job-sequencing-by-deadline example, walk through, out loud, why sorting by deadline and greedily filling the earliest available slot never needs to be reconsidered later — what specific property of the problem guarantees this?

12.3 — Interval scheduling problems

1. PLAIN-LANGUAGE GIST

Interval scheduling asks: given a set of intervals (each with a start and end time), select the maximum number of non-overlapping intervals — and the provably correct greedy strategy is to sort by end time and repeatedly take the earliest-ending interval that doesn't overlap with what's already been selected, a specific, well-justified instance of 12.1's greedy choice property that's worth knowing by name given how frequently this exact shape recurs.

2. REAL-WORLD ANALOGY

Picture booking a single conference room for as many separate meetings as possible throughout a day, where meetings can't overlap. Greedily accepting whichever pending meeting finishes earliest (among ones that don't conflict with already-accepted meetings) always leaves the most remaining time available for future meetings — a meeting that ends sooner can never be a worse choice than one that ends later, when your only goal is fitting in as many total meetings as possible.

3. CONNECT TO PRIOR KNOWLEDGE

This is a concrete, fully-justified instance of 12.1's greedy choice property — interval scheduling is one of the classic, provable greedy algorithms (mentioned by name in 12.2's deeper knowledge), and it directly reuses 7.5's “sort first” insight, since sorting by end time is precisely what unlocks the simple, correct single-pass greedy logic that follows.

4. CORE MECHANISM (Simple → Intermediate)

Simple model: Sort all intervals by their end time, ascending. Initialize “last selected end time” to negative infinity (nothing selected yet). Walk through the sorted intervals in order; for each one, if its start time is at or after the last selected interval's end time (no overlap), select it and update “last selected end time” to this interval's end.

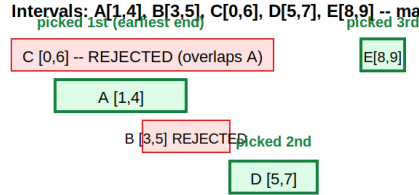
Next layer — why sorting by end time specifically (not start time, not duration) is the provably correct choice:

- **Why not sort by start time:** an interval that starts earliest might also be extremely long, blocking out a huge amount of time that could otherwise have fit several shorter, non-overlapping intervals — starting early says nothing about how much “room” a choice consumes.
- **Why not sort by shortest duration:** a short interval might be positioned awkwardly such that picking it doesn't actually free up the most useful subsequent time — duration alone doesn't capture how much remaining usable time a choice preserves.
- **Why end time specifically works:** the interval that finishes earliest, among all currently-viable candidates, guarantees the earliest possible point from which the remaining schedule can resume — no other valid choice could possibly free up an earlier resumption point, provable via an exchange argument (12.1's technique).
- **This exact pattern generalizes directly to several named variants:** “minimum number of meeting rooms needed,” “maximum number of non-overlapping intervals to remove,” and “merge overlapping intervals” (7.5's example, which uses a different subsequent merging logic rather than this topic's selection logic).

5. VISUAL REPRESENTATION

Interval scheduling: greedily pick by EARLIEST END TIME, never start time

Intervals: A[1,4], B[3,5], C[0,6], D[5,7], E[8,9] -- maximize non-overlapping count



Result: {A, D, E} -- 3 non-overlapping intervals, the maximum possible

Sort by END time first. Greedily take the first interval (earliest end), then skip every interval overlapping it, then take the next surviving interval -- repeat.

Why earliest END (not earliest START or shortest duration): finishing sooner always leaves the MOST room for future intervals -- provably never a worse choice.

The diagram traces the greedy algorithm on five intervals: C (the longest, earliest-starting) is rejected immediately because, despite starting first, it would block out the most time; A is correctly selected first since it has the earliest end time among non-rejected options, followed by D and E, producing the maximum possible count of 3 non-overlapping intervals.

6. WHEN TO USE / WHEN NOT TO USE

Use the sort-by-end-time greedy pattern when:

- The problem asks to maximize the count of non-overlapping intervals selected — the canonical, directly-applicable shape this topic addresses.
- You recognize a problem as a variant of this pattern (meeting room scheduling, interval removal) — the same sort-by-end-time foundation typically still applies.

Wrong choice when:

- The problem asks to maximize total value or weight of selected intervals rather than just the count — this is a structurally different, weighted version generally requiring DP instead.
- The problem is actually about merging overlapping intervals into combined ranges (7.5's example) rather than selecting a maximum non-overlapping subset.

7. C# EXAMPLES — Simple → Intermediate → Advanced

SIMPLE — maximum non-overlapping intervals, the exact diagram and algorithm above:

```
int MaxNonOverlapping(int[][] intervals) {
    Array.Sort(intervals, (a, b) => a[1].CompareTo(b[1])); // by END

    int count = 0;
    int lastEnd = int.MinValue;
    foreach (var interval in intervals) {
        if (interval[0] >= lastEnd) { // no overlap with last
            count++;
            lastEnd = interval[1];
        }
    }
    return count;
}
```

INTERMEDIATE — minimum number of intervals to remove to make the rest non-overlapping (the complementary framing of the same algorithm):

```
int MinRemovalsToNonOverlapping(int[][] intervals) {
    int totalCount = intervals.Length;
    int maxKept = MaxNonOverlapping(intervals); // reuse same logic
    return totalCount - maxKept; // remove everything NOT kept
}
```

This shows that a seemingly different question (“minimum to remove”) is mathematically just “total count minus the maximum that can be kept” — recognizing this reframing means you never need a separate algorithm for this variant.

ADVANCED — minimum number of meeting rooms required, using a sweep-line approach combined with a heap (9.1/9.2’s structure):

```
int MinMeetingRooms(int[][] intervals) {
    Array.Sort(intervals, (a, b) => a[0].CompareTo(b[0])); // by START

    var endTimesHeap = new PriorityQueue<int, int>(); // min-heap

    foreach (var interval in intervals) {
        if (endTimesHeap.Count > 0 && endTimesHeap.Peek() <= interval[0]) {
            endTimesHeap.Dequeue(); // a room just freed up -- reuse it
        }
        endTimesHeap.Enqueue(interval[1], interval[1]); // occupy a room
    }
    return endTimesHeap.Count; // rooms simultaneously needed
}
```

This is a meaningfully different variant: instead of selecting a subset to maximize count, every interval must be scheduled, and the question is how many concurrent “rooms” are needed — solved by sorting by start time this time, and using a min-heap (9.1) to always check whether the earliest-finishing currently-occupied room has freed up before allocating a new one, directly combining this topic’s interval-sorting foundation with 9.2’s PriorityQueue mechanics.

8. DEEPER KNOWLEDGE (Gist Only)

At an expert level, the formal proof that sorting by end time is optimal is a textbook example of an exchange argument (12.1’s technique) — assume an optimal solution exists that doesn’t pick the earliest-ending interval first; show that replacing whatever it does pick first with the earliest-ending interval can only free up equal or more subsequent time, never less, proving the greedy choice is always safe. There’s also a useful generalization worth knowing: weighted interval scheduling (where each interval has an associated value) breaks this exact greedy approach and instead requires a DP solution that, for each interval (sorted by end time), decides whether to include it by comparing “skip it” against “include it plus the best solution among non-conflicting earlier intervals” — a concrete instance of 12.2’s “greedy works for the unweighted version, DP needed once weights are introduced” diagnostic.

9. COMMON MISCONCEPTIONS AND MISTAKES

A common mistake is sorting by start time (or by duration) instead of end time for the basic maximum-count interval selection problem — this produces a structurally different, generally incorrect selection order. Another is assuming the same simple greedy approach extends directly to the weighted version of the problem — this is the exact “greedy stops working once an interacting constraint or differing value changes the problem’s structure” failure mode from 12.2. A third: confusing the “maximum non-overlapping subset” goal with the “minimum concurrent resources needed” goal (the meeting-rooms variant) — these are different questions requiring different algorithms, even though both involve the same kind of interval input data.

10. SELF-CHECK

- Why does sorting intervals by end time (rather than start time or duration) provably produce the correct greedy ordering for maximizing the count of non-overlapping intervals?
- Walk through, out loud, why “minimum removals to make the rest non-overlapping” doesn't need a separate algorithm from “maximum non-overlapping subset” — what's the mathematical relationship between the two questions?
- Why does the minimum-meeting-rooms variant need a fundamentally different bookkeeping approach (a heap tracking concurrent end times) rather than the simple running-count approach used for maximizing non-overlapping count?

12.4 — Common greedy patterns (e.g. always pick smallest/largest available option)

1. PLAIN-LANGUAGE GIST

This topic consolidates the recurring, recognizable shapes that valid greedy algorithms tend to take — sorting then picking smallest/largest available options satisfying a constraint, sorting by end time for interval-style problems (12.3), tracking a simple running best-so-far value, and ordering choices justified by an exchange argument — building a practical pattern-recognition toolkit so that encountering a new problem prompts “which of these familiar shapes might apply?” rather than reasoning about greedy validity entirely from scratch each time.

2. REAL-WORLD ANALOGY

Picture a craftsperson who has built up a toolkit of recognizable techniques for common situations — a specific way of measuring twice before cutting, a specific joint technique for corners, a specific approach for uneven surfaces — built from experience with many prior projects. When a new project arrives, the craftsperson doesn't reinvent woodworking from first principles each time; they recognize “this situation matches the kind of problem my corner-joint technique solves” and apply the proven approach, adapting details as needed. This topic builds that same kind of recognizable-template toolkit for greedy algorithm design.

3. CONNECT TO PRIOR KNOWLEDGE

This topic directly extends 12.1's greedy choice property and 12.3's worked interval-scheduling example into a broader catalog — every pattern shown here still requires 12.1's underlying validation (recognizing a familiar shape is a strong hint that greedy might apply, not a substitute for confirming the greedy choice property actually holds for the specific problem at hand), exactly the same caution 12.2's diagnostic process emphasized.

4. CORE MECHANISM (Simple → Intermediate)

Simple model: Most valid greedy algorithms fall into a small number of recurring shapes: (1) sort, then greedily satisfy a constraint with the smallest/largest sufficient option (assigning cookies to children); (2) sort by a specific derived criterion (often end time) for interval-style problems (12.3's fully-worked pattern); (3) maintain a simple running best-so-far value, taking every locally beneficial opportunity (accumulating every positive price difference in a stock-trading problem); (4) order choices according to a property justified by an exchange argument, even when the “obvious” ordering criterion isn't immediately apparent from the problem statement alone.

Next layer — recognizing which specific pattern a new problem matches, and the recurring validation requirement:

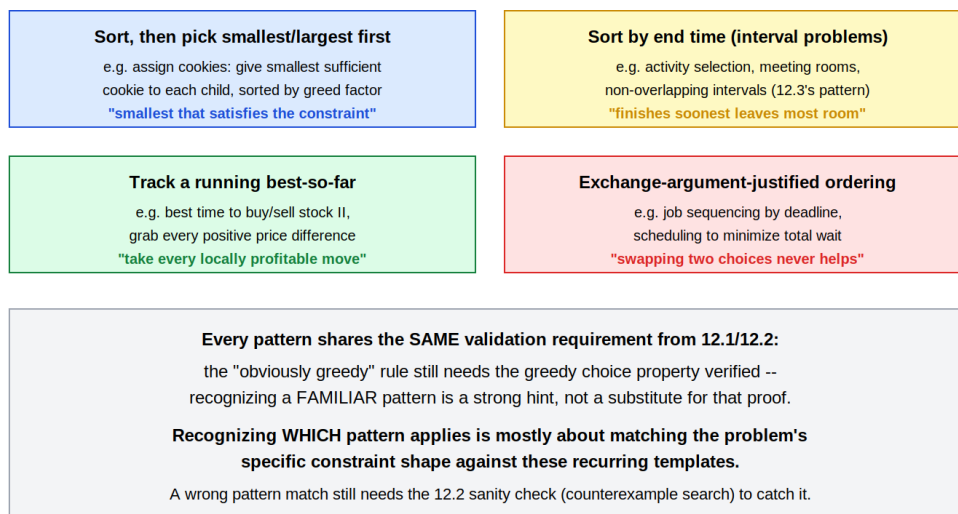
- **Pattern 1** (smallest/largest sufficient option) typically applies to “matching” or “assignment” problems with a clear satisfiability constraint — sorting both the “requesters” and the “available options” lets a single coordinated pass match them efficiently.
- **Pattern 2** (sort by end time) is specifically interval/scheduling-shaped, as fully justified in 12.3 — recognizing “this problem is fundamentally about non-overlapping time ranges” is the trigger for trying this specific, well-proven template first.
- **Pattern 3** (running best-so-far) applies when the problem's optimal answer can be decomposed into a sum (or accumulation) of independent, locally-decidable contributions — each “take this opportunity” decision doesn't affect the validity of any other opportunity, unlike 11.6's knapsack where choices compete for shared, limited capacity.

- **Pattern 4** (exchange-argument-justified ordering) is the most general, and the one most likely to require actually constructing the proof rather than just recognizing a template — problems like minimizing total waiting time in a scheduling sequence often have a non-obvious “sort by this specific derived quantity” rule that only becomes apparent after attempting an exchange argument directly.

- **The validation requirement never goes away:** recognizing that a problem resembles one of these patterns is a strong, useful heuristic for where to start, but 12.1's underlying verification is still the deciding factor — pattern-matching accelerates the search for a correct approach, it doesn't replace the need to confirm correctness.

5. VISUAL REPRESENTATION

Common greedy patterns: the recurring "always pick X" templates



The diagram presents four recurring greedy pattern templates side by side — sort-then-match, sort-by-end-time, running-best-so-far, and exchange-argument-justified ordering — each labeled with a representative example problem and its core justification, with the unifying note that every pattern still requires confirming 12.1's greedy choice property for the specific problem at hand, rather than relying on the pattern match alone.

6. WHEN TO USE / WHEN NOT TO USE

Use pattern-matching as a starting heuristic when:

- You encounter a new problem and want an efficient way to generate a candidate approach quickly — recognizing “this looks like an interval-scheduling shape” or “this looks like a sort-and-match shape” gives you a concrete starting template to test, rather than starting from a blank page.

Don't treat pattern-matching as sufficient validation when:

- You've identified a resembling pattern but haven't actually confirmed the greedy choice property holds for this specific problem's exact constraints — always follow up with 12.1/12.2's verification process.
- The problem has an interacting constraint (shared, limited capacity competed for by multiple choices, as in 11.6's knapsack) that breaks the “each choice is independently safe” assumption underlying most of these greedy templates — this is a strong signal to reconsider whether DP (12.2) is actually the better-fitting tool.

7. C# EXAMPLES — Simple → Intermediate → Advanced

SIMPLE — Pattern 1 (sort, then match smallest sufficient option), the “assign cookies” problem:

```
int AssignCookies(int[] childGreed, int[] cookieSizes) {
    Array.Sort(childGreed);
    Array.Sort(cookieSizes);

    int childIndex = 0, cookieIndex = 0, satisfied = 0;
    while (childIndex < childGreed.Length && cookieIndex < cookieSizes.Length) {
        if (cookieSizes[cookieIndex] >= childGreed[childIndex]) {
            satisfied++;          // this cookie satisfies this child
            childIndex++;
        }
        cookieIndex++;           // move to next cookie regardless
    }
    return satisfied;
}
```

Sorting both lists lets a single coordinated pass match the least-demanding child with the smallest sufficient cookie — giving a more-demanding child a cookie that a less-demanding child could have used “wastes” that cookie’s larger size on a need that didn’t require it.

INTERMEDIATE — Pattern 3 (running best-so-far), maximizing profit from unlimited stock trades:

```
int MaxProfitUnlimitedTrades(int[] prices) {
    int totalProfit = 0;
    for (int i = 1; i < prices.Length; i++) {
        if (prices[i] > prices[i - 1]) {
            totalProfit += prices[i] - prices[i - 1]; // capture gain
        }
    }
    return totalProfit;
}
```

Each day-to-day price increase can be captured independently of every other one (buy the day before any rise, sell the day of), with no shared, limited resource being competed for between these decisions — exactly Pattern 3’s “each opportunity is independently safe to take” signature, distinguishing it from constrained problems (like a limited number of trades allowed) which would reintroduce interacting constraints and likely require DP instead.

ADVANCED — Pattern 4 (exchange-argument-justified ordering), minimizing total waiting time when scheduling jobs on a single processor:

```
// Minimize the SUM of completion times across all jobs -- the
// non-obvious greedy rule: sort by SHORTEST DURATION FIRST
int MinTotalCompletionTime(int[] durations) {
    Array.Sort(durations); // shortest first -- the key insight

    int totalCompletionTime = 0, currentTime = 0;
    foreach (int duration in durations) {
        currentTime += duration;
        totalCompletionTime += currentTime;
    }
    return totalCompletionTime;
}
// WHY shortest-first: if a longer job runs before a shorter one,
// swapping their order strictly reduces total completion time --
// a textbook exchange argument: any adjacent "long-then-short" pair
// can always be improved by swapping, so the optimal order must have
// no such pair anywhere, meaning everything must be sorted shortest-first.
```


This is deliberately the least “obvious” pattern of the four — there’s no immediately intuitive reason to suspect “shortest first” without actually working through the exchange argument’s swap-and-compare logic, making it a good illustration of Pattern 4’s defining trait: the correct greedy ordering criterion sometimes needs to be derived through the proof process itself, rather than being visible from the problem statement at a glance.

8. DEEPER KNOWLEDGE (Gist Only)

At an expert level, recognizing these recurring greedy shapes is itself an instance of a broader skill in algorithm design called pattern-based problem solving — building a mental library of “problem shape → proven technique” associations that accelerates recognizing which broad category of technique to attempt first on a new, unfamiliar problem. There’s also a valuable connection worth flagging explicitly: several of the most celebrated classical greedy algorithms — Huffman coding, Kruskal’s and Prim’s algorithms (minimum spanning trees, Tier 2/3 topics) — are essentially sophisticated instances of Pattern 4, where the non-obvious correct ordering criterion was itself a significant historical discovery, reinforcing that this pattern category rewards genuine creative problem-solving rather than rote template application.

9. COMMON MISCONCEPTIONS AND MISTAKES

A common mistake is treating pattern recognition as a complete substitute for verification — correctly identifying “this resembles an interval-scheduling problem” doesn’t automatically guarantee the specific problem at hand shares every assumption the original proven pattern relies on. Another is assuming Pattern 3’s “running best-so-far” shape applies whenever a problem involves accumulating a value over a sequence, without checking whether a shared, limited constraint actually exists, which would reintroduce exactly the interacting-choices problem that breaks this pattern’s safety assumption. A third: giving up on finding a valid greedy approach too early when a problem doesn’t match any of the three more “obvious” patterns (1-3), without attempting Pattern 4’s harder, more creative exchange-argument derivation.

10. SELF-CHECK

- For each of the four patterns described in this topic, identify the specific structural feature of a problem that would signal “try this pattern first.”
- Why does Pattern 3’s “running best-so-far” approach work correctly for unlimited stock trading, but would likely fail (or need significant modification) if the problem instead limited you to at most two total trades?
- In the shortest-job-first scheduling example, walk through, out loud, the core exchange-argument logic that justifies sorting by duration — what specifically happens to total completion time when you swap an adjacent “long job, then short job” pair?